Parallel Multiplier

For the XOR gate, I used RTL because it is much simpler and easier to read. With complicated designs, unnecessary challenges may occur when using structural design for the XOR gate.

In simulation there is no delay, but considering the critical path delay, the longest path can be found for a calculation to be completed. The steps for the multiplication include generating the partial products from AND gates, secondly the carry-save addition where we add the partial products to each-other and this happens twice until we finally have a ripple-carry addition for the final step which will give the final product.

At first the design included ripple carry for all stages of the multiplier for simplicity to see how the basic level multiplication worked. Looking at the critical path, where each gate has t delay, worst case delay can be calculated:

1. t delay for AND gates formulating the partial products since they all are done in parallel
2. 16t delay for the addition of (pp0 + pp1) [8 full adders each with 2t delay]
3. 16t delay for second addition (sum1 + pp2)
4. 16t delay for final addition (sum2 + pp3)

```
    -- Stage 1: sum1 = pp0 + pp1

    FA10: full_adder port map(pp0(0), pp1(0), '0',    sum1(0), carry1(0));
    FA11: full_adder port map(pp0(1), pp1(1), carry1(0), sum1(1), carry1(1));
    FA12: full_adder port map(pp0(2), pp1(2), carry1(1), sum1(2), carry1(2));
    FA13: full_adder port map(pp0(3), pp1(3), carry1(2), sum1(3), carry1(3));
    FA14: full_adder port map(pp0(4), pp1(4), carry1(3), sum1(4), carry1(4));
    FA15: full_adder port map(pp0(5), pp1(5), carry1(4), sum1(5), carry1(5));
    FA16: full_adder port map(pp0(6), pp1(6), carry1(5), sum1(6), carry1(6));
    FA17: full_adder port map(pp0(7), pp1(7), carry1(6), sum1(7), carry1(7));


    -- Stage 2: sum2 = sum1 + pp2

    FA20: full_adder port map(sum1(0), pp2(0), '0', sum2(0), carry2(0));
    FA21: full_adder port map(sum1(1), pp2(1), carry2(0), sum2(1), carry2(1));
    FA22: full_adder port map(sum1(2), pp2(2), carry2(1), sum2(2), carry2(2));
    FA23: full_adder port map(sum1(3), pp2(3), carry2(2), sum2(3), carry2(3));
    FA24: full_adder port map(sum1(4), pp2(4), carry2(3), sum2(4), carry2(4));
    FA25: full_adder port map(sum1(5), pp2(5), carry2(4), sum2(5), carry2(5));
    FA26: full_adder port map(sum1(6), pp2(6), carry2(5), sum2(6), carry2(6));
    FA27: full_adder port map(sum1(7), pp2(7), carry2(6), sum2(7), carry2(7));


    -- Stage 3: P = sum2 + pp3 Ripple carry

    FA30: full_adder port map(sum2(0), pp3(0), '0', P(0), carry3(0));
    FA31: full_adder port map(sum2(1), pp3(1), carry3(0), P(1), carry3(1));
     FA32: full_adder port map(sum2(2), pp3(2), carry3(1), P(2), carry3(2));
    FA33: full_adder port map(sum2(3), pp3(3), carry3(2), P(3), carry3(3));
    FA34: full_adder port map(sum2(4), pp3(4), carry3(3), P(4), carry3(4));
    FA35: full_adder port map(sum2(5), pp3(5), carry3(4), P(5), carry3(5));
    FA36: full_adder port map(sum2(6), pp3(6), carry3(5), P(6), carry3(6));
    FA37: full_adder port map(sum2(7), pp3(7), carry3(6), P(7), carry3(7));

end Structural;
```

For a total of 49t, however when a carry save method is used instead, we get delays of:

1. t delay for AND gates generating partial products
2. 2t for pp0 +pp1 since CSA eliminates the carry propagation per row
3. 2t for sum1 +pp2

4. 16t for the final ripple addition
   For a total of 21t

Using Carry-save adders will reduce the delay by 42.8% which is a huge margin. Without CSA, you would be using less adders so if you do not care about delays as much then there is an argument that you may not want to use this method to reduce area size.

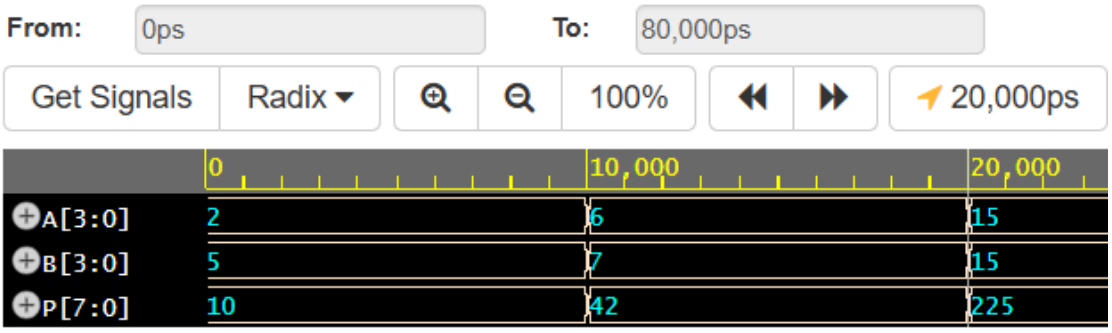

Figure 1: Simulation results

As we can see from figure 1, we get the expected results for multiplication. In previous designs, when simulating with different testbench values, the calculations were all correct except 15*15. This was due to an overflow problem which was addressed which is why 15*15 was always tested for each design. When simulating we can see that the simulation took 20ns.

https://www.edaplayground.com/x/WxLg